



**Calculador de Tarifas, Presupuestos y Descuentos en NipinFinan: Integración de
JavaScript y Estructura EORM**

Santiago Gonzalez

Facultad de Ingeniería, Tecnología y Arquitectura, Universidad de la Cuenca del Plata

Paradigmas y Lenguajes de Programación III, Comisión "A"

Ing. Sicardi Dante

29 de septiembre de 2026



Calculador de Tarifas, Presupuestos y Descuentos en NipinFinan: Integración de JavaScript y Estructura EORM

NipinFinan es un sistema web de gestión y análisis de economía personal que permite registrar ingresos y gastos, organizarlos por categoría y controlar el dinero disponible. En la primera etapa del trabajo (TP1 y AE1) se realizó la maquetación de las pantallas principales con HTML5 semántico y CSS3. En esta segunda actividad evaluativa (AE2) se incorporó JavaScript para agregar comportamiento dinámico a uno de los módulos del sitio, manteniendo la temática original de finanzas personales.

El objetivo de este informe es describir cómo se resolvió el reto asignado, que consiste en un calculador de tarifas, presupuestos y descuentos integrado al formulario de registro de movimientos, y fundamentar el diseño de la tarifa y la promoción mediante la estructura EORM trabajada en la Unidad 1. También se detallan las pruebas que se hicieron para verificar que la solución cumple con la consigna.

El código fuente se encuentra en la rama *ae2-gonzalez_santiago* del repositorio del proyecto (Fohrholtz y Gonzalez, 2026), disponible en https://github.com/LuchoFohrholtz/NipinFinan/tree/ae2-gonzalez_santiago. La entrega corresponde a esa rama y no a la rama principal (*main*), ya que en paralelo se está desarrollando otra rama del proyecto y por el momento no se van a integrar.

Consigna y Tarjeta Asignada

La consigna establece un requisito transversal mínimo para todos los grupos: los eventos deben capturarse con `addEventListener()` desde `app.js`, sin usar atributos como `onclick` u `onsubmit` en el HTML, y los datos deben cargarse de forma asíncrona con la API `fetch()` y la sintaxis `async/await` a partir de un archivo JSON local.

Al grupo le correspondió la Opción 5, Calculador de Tarifas, Presupuestos y Descuentos. El reto pide escuchar los cambios en la cantidad o el tipo de servicio con



addEventListener('change'), obtener la tabla de precios y descuentos desde un archivo JSON, calcular el presupuesto de forma condicional y mostrar el total actualizado en el DOM. Como fundamentación de la Unidad 1 se pide documentar la estructura EORM, es decir, los atributos y el comportamiento de la tarifa y la promoción. En la Tabla 1 se resume cómo se cumplió cada punto.

Tabla 1

Requisitos de la Consigna y su Implementación en el Proyecto

Requisito	Implementación	Archivo
Desacoplamiento de eventos	Todos los eventos se registran con addEventListener(); ningún HTML tiene atributos on*	js/app.js
Asincronía con fetch()	cargarTarifas() usa fetch() con async/await y manejo de errores	js/app.js
Escucha de cambios	'change' en tipo de movimiento, medio de pago, cuotas y monto	js/app.js
Tabla de precios en JSON	Descuentos, intereses por cuotas y cupones	data/tarifas.json
Cálculo condicional	calcular() aplica reglas distintas según tipo, medio de pago y cuotas	js/app.js
Actualización del DOM	actualizarResultado() muestra subtotal, ajuste, total y cuota	comprar.html
Estructura EORM	Atributos, comportamiento y relaciones documentados	README.md e informe

Nota. Elaboración propia a partir de la consigna de la AE2.

Desarrollo

El módulo elegido para enriquecer con JavaScript fue la pantalla Registrar movimiento (comprar.html), porque es donde el usuario carga un gasto y elige el medio de



pago, lo que permite aplicar descuentos e intereses de manera natural dentro de la temática del proyecto.

Desacoplamiento de Eventos

Para separar la estructura (HTML) del comportamiento (JavaScript), el formulario no tiene ningún atributo de evento. Los manejadores se registran con `addEventListener()`, que permite asociar una o más funciones a un evento de un elemento sin modificar el marcado (Mozilla Developer Network [MDN], s.f.-b). Los listeners se agregan dentro del evento `DOMContentLoaded`, después de cargar las tarifas, como se ve en la Figura 1.

Figura 1

Registro de Eventos del Formulario con `addEventListener()`

```
document.addEventListener("DOMContentLoaded", async () => {  
    inputFecha.value = fechaHoy();  
    await cargarTarifas();  
    actualizarResultado();  
  
    inputMonto.addEventListener("input", actualizarResultado);  
    inputMonto.addEventListener("change", actualizarResultado);  
    selectPago.addEventListener("change", actualizarResultado);  
    selectCuotas.addEventListener("change", actualizarResultado);  
    radiosTipo.forEach(radio =>  
        radio.addEventListener("change", actualizarResultado));  
  
    form.addEventListener("submit", manejarSubmit);  
});
```

Nota. Fragmento de `js/app.js` (Gonzalez, 2026). En el monto se escucha también el evento `input` para que el total se actualice mientras el usuario escribe.

Carga Asíncrona de Tarifas

La tabla de precios no está escrita en el código, sino en el archivo `data/tarifas.json`. Al iniciar la página, la función `cargarTarifas()` la obtiene con `fetch()`, que devuelve una promesa con la respuesta del servidor (MDN, s.f.-c). Gracias a `async/await` el código asíncrono se lee de forma secuencial (MDN, s.f.-a). Se controla la propiedad `ok` de la respuesta y, si ocurre un error, se informa al usuario y se deshabilita el botón Guardar para no registrar totales incorrectos (ver Figura 2).



Figura 2

Función cargarTarifas() con fetch() y async/await

```
async function cargarTarifas() {  
  try {  
    const respuesta = await fetch("data/tarifas.json");  
    if (!respuesta.ok) throw new Error("HTTP " + respuesta.status);  
    tarifas = await respuesta.json();  
  } catch (err) {  
    console.error("Error al cargar tarifas:", err);  
    mostrarMensaje("No se pudieron cargar las tarifas. ...",  
"error");  
    btnGuardar.disabled = true;  
  }  
}
```

Nota. Fragmento de js/app.js (Gonzalez, 2026).

El contenido del archivo JSON se resume en la Tabla 2. Tener los valores separados permite cambiar un descuento o un interés sin tocar el código JavaScript.

Tabla 2

Contenido del Archivo data/tarifas.json

Sección	Clave	Valor
medios_pago	Efectivo	10 % de descuento
medios_pago	Transferencia	5 % de descuento
medios_pago	Tarjeta de débito / Mercado Pago / Tarjeta de crédito	Sin descuento
cuotas	1 / 3 / 6 / 12	0 % / 8 % / 18 % / 35 % de interés
cupones	UCP10	10 % de descuento, vigente
cupones	VERANO25	25 % de descuento, vencido

Nota. Los porcentajes se guardan como decimales en el JSON (por ejemplo, 0.10 equivale a 10 %).

Cálculo Condicional del Presupuesto

La función calcular() lee los valores actuales del formulario y aplica reglas distintas según el caso. Si el movimiento es un ingreso, no se aplica ningún descuento ni cuotas. Si es un gasto, se aplica el descuento que corresponde al medio de pago elegido. Si además el



medio de pago es tarjeta de crédito, se habilita el selector de cuotas y se suma el interés del plan elegido. El total se obtiene con la siguiente fórmula:

$$total = monto \times (1 - descuento) \times (1 + interés)$$

El valor de cada cuota es el total dividido por la cantidad de cuotas. Los resultados se redondean a dos decimales. La parte central de la función se muestra en la Figura 3.

Figura 3

Lógica Condicional de la Función calcular()

```
if (tipoSeleccionado() === "ingreso") {  
    return { subtotal: monto, ajuste: 0, total: monto,  
            cuotas: 1, valorCuota: monto };  
}  
  
const medio = selectPago.value;  
const esCredito = medio === "Tarjeta de crédito";  
const cuotas = esCredito ? parseInt(selectCuotas.value, 10) : 1;  
  
const descuento = tarifas?.medios_pago?.[medio]?.descuento ?? 0;  
const interes = esCredito ? (tarifas?.cuotas?.[cuotas] ?? 0) : 0;  
  
const base = monto * (1 - descuento);  
const total = redondear(base * (1 + interes));
```

Nota. Fragmento de js/app.js (Gonzalez, 2026).

Actualización del DOM y Persistencia

Cada vez que cambia algún dato, actualizarResultado() vuelve a calcular y escribe en la sección Resumen el subtotal, el descuento o recargo, el total y, si hay más de una cuota, el valor de cada cuota. Para escribir se usa `textContent`, que no interpreta HTML y evita que un texto ingresado por el usuario se ejecute como código.

Al presionar Guardar se valida que haya descripción, monto mayor a cero, fecha, categoría y medio de pago. Si todo es correcto, el movimiento se guarda en `localStorage`, que conserva los datos en el navegador aunque se cierre la página (MDN, s.f.-d), y el formulario se limpia.



Módulos Complementarios

Para que la calculadora se integre con el resto del sitio se completaron dos pantallas más. En listado_tabla.html el archivo js/listado.js lee los movimientos guardados, los ordena por fecha y calcula los totales de ingresos, gastos y dinero disponible. En cupones.html el archivo js/cupones.js valida códigos de descuento leyéndolos, también con fetch(), desde la sección cupones del mismo archivo JSON, de modo que todas las promociones salen de una única fuente de datos.

Fundamentación U1: Estructura EORM

Para documentar la tarifa y la promoción se las modeló como objetos, cada uno con un estado formado por sus atributos y un comportamiento dado por las operaciones que se pueden realizar sobre ellos (Booch et al., 2007). Los atributos corresponden a los datos que llegan desde el archivo JSON y el comportamiento a las funciones de JavaScript que los utilizan. Este tipo de modelado ayuda a separar los datos de la lógica y a identificar las responsabilidades de cada parte del sistema (Pressman y Maxim, 2020).

Atributos

En la Tabla 3 se detallan las entidades identificadas y sus atributos.



Tabla 3

Entidades de la Tarifa y la Promoción con sus Atributos

Entidad	Atributo	Tipo	Ejemplo
Tarifa	medios_pago	Objeto	Colección de medios de pago
Tarifa	cuotas	Objeto	Colección de planes de cuotas
Tarifa	cupones	Objeto	Colección de promociones
MedioPago	nombre	Cadena	"Efectivo"
MedioPago	descuento	Número	0.10
PlanCuotas	cantidad	Número	12
PlanCuotas	interés	Número	0.35
Promoción (cupón)	código	Cadena	"UCP10"
Promoción (cupón)	descuento	Número	0.10
Promoción (cupón)	vigente	Booleano	true

Nota. Elaboración propia a partir de data/tarifas.json.

Comportamiento

Las operaciones que definen el comportamiento de la tarifa y la promoción se presentan en la Tabla 4.



Tabla 4

Operaciones que Definen el Comportamiento de la Tarifa y la Promoción

Operación	Archivo	Responsabilidad
cargarTarifas()	js/app.js	Obtener la tabla de tarifas con fetch()
calcular()	js/app.js	Aplicar descuento e interés según tipo, medio de pago y cuotas
mostrarCuotasSiCorresponde()	js/app.js	Mostrar las cuotas solo en un gasto con tarjeta de crédito
actualizarResultado()	js/app.js	Escribir subtotal, ajuste, total y valor de cuota en el DOM
validarCupon()	js/cupones.js	Verificar si un cupón existe y está vigente

Nota. Elaboración propia.

Relaciones

La Tarifa agrupa todos los medios de pago, los planes de cuotas y las promociones. Un movimiento de tipo gasto se relaciona con un único medio de pago y, cuando ese medio es tarjeta de crédito, también con un plan de cuotas. Un movimiento de tipo ingreso no se relaciona con ninguna tarifa, ya que no lleva descuentos ni intereses.

Pruebas Realizadas

Para verificar el funcionamiento se ejecutaron pruebas automatizadas en el navegador Google Chrome, con el sitio servido desde un servidor local. En total se ejecutaron 53 casos y todos resultaron correctos. En la Tabla 5 se agrupan por área y en la Tabla 6 se muestran algunos ejemplos de cálculo verificados.



Tabla 5

Resumen de las Pruebas Automatizadas

Área	Qué se verificó	Casos
Requisitos generales	Ausencia de atributos on* en los HTML; uso de fetch, await y 'change'	8
Calculadora	Carga del JSON, descuentos, cuotas, ingresos y montos con decimales	21
Validaciones	Mensajes de error por campos incompletos	2
Guardado	Mensaje de éxito, limpieza del formulario y datos en localStorage	4
Listado	Filas, totales, orden, aviso sin datos y texto con HTML no ejecutado	7
Cupones	Cupón válido, vencido, inexistente y campo vacío	4
Navegación	Carga de las seis páginas y menú completo	6
Manejo de errores	Mensaje y botón deshabilitado si falla fetch()	1
Total		53

Nota. Elaboración propia.



Tabla 6

Ejemplos de Cálculo Verificados con un Monto de \$10.000

Tipo	Medio de pago	Cuotas	Total	Valor de cuota
Gasto	Efectivo	1	\$9.000	—
Gasto	Transferencia	1	\$9.500	—
Gasto	Tarjeta de débito	1	\$10.000	—
Gasto	Tarjeta de crédito	3	\$10.800	\$3.600
Gasto	Tarjeta de crédito	12	\$13.500	\$1.125
Ingreso	Transferencia	1	\$10.000	—

Nota. Los ingresos no reciben descuento aunque el medio de pago lo tenga.

Instrucciones de Ejecución

Para ejecutar el proyecto se debe clonar el repositorio y cambiar a la rama `ae2-gonzalez_santiago` con el comando `git checkout ae2-gonzalez_santiago`. Luego hay que abrir la carpeta con un servidor local, por ejemplo la extensión Live Server de Visual Studio Code, e ingresar a `comprar.html`. Es importante no abrir el archivo HTML con doble clic, porque en ese caso el navegador bloquea la petición `fetch()` al archivo JSON y la calculadora no puede cargar las tarifas.

Conclusión

Con esta actividad se logró que el formulario de registro de NipinFinan deje de ser estático y calcule en tiempo real el costo de un gasto según el medio de pago y las cuotas. Separar la tabla de tarifas en un archivo JSON y registrar los eventos desde JavaScript hizo que el código sea más ordenado y fácil de modificar, ya que para cambiar un descuento solo hace falta editar un dato. Además, modelar la tarifa y la promoción con la estructura EORM permitió tener claro qué datos maneja cada entidad y qué operaciones las usan antes de escribir el código.



Referencias

Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J. y Houston, K. A.

(2007). *Object-oriented analysis and design with applications* (3.^a ed.). Addison-Wesley.

Fohrholtz, L. y Gonzalez, S. (2026). *NipinFinan* (rama ae2-gonzalez_santiago) [Código

fuelle]. GitHub. https://github.com/LuchoFohrholtz/NipinFinan/tree/ae2-gonzalez_santiago

Mozilla Developer Network. (s.f.-a). *async function*. MDN Web Docs. Recuperado el 28 de septiembre de 2026, de

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/async_function

Mozilla Developer Network. (s.f.-b). *EventTarget.addEventListener()*. MDN Web Docs.

Recuperado el 28 de septiembre de 2026, de

<https://developer.mozilla.org/es/docs/Web/API/EventTarget/addEventListener>

Mozilla Developer Network. (s.f.-c). *Uso de Fetch*. MDN Web Docs. Recuperado el 28 de septiembre de 2026, de

https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

Mozilla Developer Network. (s.f.-d). *Window.localStorage*. MDN Web Docs. Recuperado el 28 de septiembre de 2026, de

<https://developer.mozilla.org/es/docs/Web/API/Window/localStorage>

Pressman, R. S. y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill Education.